

Universidad Nacional Autónoma de México
Facultad de Ciencias

Sistemas Operativos

Práctica 2

Syscalls y APIs en un RTOS de misión crítica para un marcapasos VVI

Profesor José León Cotonieto

Alumnos:

Rivera Morales David
Montes de Oca Aguilar Yessica Vianney

Link del repositorio

<https://github.com/MxDui/socompartido/tree/master/practica-2>

3 Caso de estudio

De: Dirección de Ingeniería y Desarrollo

Para: Equipo de Desarrollo de Firmware RTOS (R&D)

Fecha: 24 de Agosto de 2026, 08:30 CST

Asunto: Avances semanales y arquitectura de servicios/syscalls para firmware VVI

Hola equipo,

Hablé con el equipo de soporte de plataforma y ya modularizaron el código de arranque y diagnóstico (*bootstrapping*) de la semana pasada. Me comentaron que les enviarán los archivos de cabecera e implementación en el transcurso del día; procuremos mantener en todo momento el código desacoplado y modular en nuestras ramas.

Por otra parte, la División de Investigación Clínica y Sistemas Biomédicos nos solicita avances críticos para esta misma semana en la separación de privilegios del RTOS. Les reenvío íntegro el correo que nos hicieron llegar con los requerimientos técnicos.

Saludos cordiales,

Líder Técnico de Proyecto

—— Mensaje Reenviado ——

De: División de Investigación Clínica y Sistemas Médicos

Para: Dirección de Ingeniería y Desarrollo

Fecha: 23 de Agosto de 2026, 17:15 CST

Asunto: RE: Requerimientos de arquitectura de kernel y servicios para firmware VVI

Estimado equipo de desarrollo,

Para la siguiente etapa de desarrollo del marcapasos VVI sobre el ESP32, requerimos que el equipo establezca formalmente una capa de llamadas al sistema (Syscalls) y servicios de kernel que aisle completamente la lógica médica del acceso directo a los registros de hardware y periféricos.

Es indispensable contar con un mecanismo de despacho concurrente y protegido que atienda las operaciones críticas. El sistema debe ofrecer servicios protegidos para la consulta de tiempo con precisión de microsegundos, la captura y sincronización estricta de las ondas R ventriculares intrínsecas (evitando bloqueos indefinidos o carreras críticas), y la emisión de pulsos de estimulación con validación en el kernel para rechazar duraciones de pulso que se encuentren fuera de la ventana terapéutica segura (100 μ s a 2000 μ s).

Asimismo, se debe centralizar el registro de telemetría médica en un búfer circular protegido y garantizar la alimentación periódica del Watchdog de hardware exclusivamente a través de la interfaz del supervisor para evitar falsos disparos de seguridad.

Adjuntamos un documento con los detalles técnicos requeridos para que procedan a desglosar los requerimientos funcionales, no funcionales y las restricciones de diseño previas a la implementación.

Atentamente,

Investigación y Desarrollo Biomédico

Documento Adjunto: Especificación Técnica de Servicios de Kernel y Syscalls (VVI-SYS-SPEC-v2.1)

Resumen Ejecutivo del Subsistema de Servicios

El objetivo del subsistema es establecer una barrera de contención de fallos entre las tareas de control médico (espacio de aplicación) y los periféricos de estimulación, sensado y temporización (espacio de kernel) en el microcontrolador ESP32 bajo FreeRTOS.

1. Contexto Operativo del Marcapasos VVI

El marcapasos en modo VVI opera en el ventrículo y responde bajo una lógica de estimulación a demanda:

- **Intervalo de Escape Ventricular (T_{escape}):** Frecuencia base garantizada de 60 ppm ($T_{\text{escape}} = 1000$ ms). Si no se detecta actividad intrínseca en este intervalo, se debe forzar una estimulación ventricular.
- **Periodo Refractario Ventricular (VRP):** Intervalo posterior a un sensado o estimulación ($VRP = 250$ ms) durante el cual el sistema ignora cualquier ruido eléctrico para evitar sobre-sensado de la onda T.
- **Ventana de Sensado Activa:** Periodo de escucha activa equivalente a $T_{\text{escape}} - VRP = 750$ ms.

2. Requerimientos de la Capa de Syscalls y Servicios

ID Servicio	Servicio Solicitado	Criterios Técnicos y Restricciones
SYS_GET_TIME	Temporización de alta resolución	Retorno de tiempo monótonico en microsegundos (μs) sin desbordamiento temprano. Debe ser seguro ante accesos concurrentes.
SYS_WAIT_SENSE	Sincronización de sensado ventricular	Bloqueo eficiente de la tarea médica en espera de una despolarización ventricular (onda R). La notificación debe originarse en la ISR asociada al pin de sensado sin introducir latencias mayores a $50 \mu s$.
SYS_PACE_PULSE	Generación de pulso de estimulación	Validación obligatoria en espacio de kernel: rechazar pulsos con ancho $< 100 \mu s$ o $> 2000 \mu s$. La generación física del pulso debe ser atómica y no interrumpible por tareas de menor prioridad.
SYS_LOG_EVENT	Registro y telemetría	Transferencia de cadenas de texto informativas y marcas temporales hacia el puerto serie o búfer circular sin bloquear la ejecución de la tarea médica.
SYS_KICK_WDT	Refresco del Watchdog	Alimentación del temporizador de vigilancia (<i>Task WDT</i>) ejecutada exclusivamente por el hilo supervisor tras validar el estado saludable de la aplicación.

3. Restricciones de Arquitectura y Concurrency

1. **Desacoplamiento Espacial y de Privilegio:** Las tareas de aplicación no tienen permitido manipular directamente los registros de GPIO ni las estructuras internas de temporizadores. Toda interacción debe canalizarse a través de una cola de solicitudes (*Syscall Request Queue*) despachada por un hilo supervisor central (*KernelService*) con prioridad máxima (`configMAX_PRIORITIES - 1`).
2. **Sincronización Inter-Contextos:** La sincronización entre la rutina de interrupción de hardware (ISR) y la capa de servicios debe efectuarse mediante semáforos binarios o banderas de notificación optimizadas para ISR (*FromISR*).
3. **Determinismo Temporal:** El tiempo de atención de una solicitud de estimulación desde su emisión hasta el flanco de subida en el pin de salida no debe superar los 100 μ s.
4. **Tolerancia a Fallos:** En caso de recibir parámetros inválidos en cualquier servicio (ej. ancho de pulso fuera de rango o puntero nulo en registro), el kernel debe registrar la anomalía, rechazar la operación y retornar un código de estado explícito sin provocar el reinicio o pánico del sistema.

4 Desarrollo

4.1 Análisis de requerimientos y diseño arquitectónico

Antes de escribir cualquier línea de código, se debe estructurar formalmente el modelo mental del sistema. Esta etapa define las fronteras entre el espacio de aplicación y el espacio de kernel.

1. Extracción y clasificación de requerimientos:

- Realice una lectura detallada del comunicado interno y del documento de especificación adjunto.  Requerimiento No funcional  Requerimiento funcional
- Subraye e identifique de forma explícita los **requerimientos funcionales** (servicios solicitados, entradas, salidas y comportamientos esperados).

- SYS_GET_TIME Servicio solicitud: temporización a altas resoluciones. Entrada: ninguna. Salida: tiempo monotónico en microsegundos (`uint64_t`). Comportamiento esperado: No debe desbordarse prematuramente.
- SYS_WAY_SENSE Servicio solicitud: sincronización de sentido ventricular. Entrada: tiempo de espera. Salida: booleano (Sensado). Comportamiento esperado: bloquear eficientemente la tarea médica hasta detectar una ronda R o agotar el time out, notificado desde la ISR al pin sensado.
- SYS_PACE_PULSE Servicio solicitud: generación de pulso de estimulación. Entrada: ancho de pulso en microsegundos. Salida: booleano (Exito/Rechazo). Comportamiento esperado: validar el ancho del espacio del Kernel y generar el pulso físico si es válido.
- SYS_LOG_EVENT Servicio solicitud: registro y lectura. Entrada: cadena de texto. Salida: ninguna. Comportamiento esperado: transferir el mensaje con marca de tiempo hacia el puerto serie o buffer circular sin bloquear la tarea.
- SYS_KICK_WDT Servicio solicitud: Refresco del Watchdog. Entrada: ninguna. Salida: ninguna. Comportamiento esperado: aumentar el temporizador de vigilancia solo cuando el hilo supervisor confirma que la aplicación médica sigue activa.
- Identifique y clasifique los **requerimientos no funcionales** y restricciones operativas (latencias máximas, determinismo temporal, ventanas de pulso válidas, manejo de errores y contención de fallos).

Concurrency: SYS_GET debe ser seguro ante accesos concurrentes

Latencia de sensado: notificación ISR $\rightarrow$ tarea en $\leq 50 \mu s$

ventana de pulso válida: $100 - 2000 \mu s$; fuera de rango rechaza

Atomicidad del pulso: no interrumpible por tareas de menor prioridad

No bloquea en logging: SYS-LOG-EVENT no debe frenar la tarea médica

Privilegio del Watchdog: solo lo alimenta el hilo supervisor y solo si la app sigue viva

Aislamiento del hardware: la app no accede a GPIO ni temporizadores; todo pasa por la cola despachada por KernelService a prioridad máxima

Sincronización ISR-Kernel: mediante semáforos binarios o notificaciones FromISR

Determinismo: solicitud de pulso $\rightarrow$ flanco de salida en $\leq 100 \mu s$

Tolerancia a fallos: parámetro inválido $\rightarrow$ registrar anomalía y rechazar, sin reinicio ni panic

2. Delimitación del Modelo de privilegio (espacio de usuario vs. espacio de kernel):

- ¿Por qué la tarea médica (vvi_controller_task) no debe acceder directamente a funciones de bajo nivel como digitalWrite(), attachInterrupt() o registros directos del microcontrolador?

Porque rompería el único punto de validación del sistema: un error en la app actuaría de inmediato sobre el hardware, sin control ni auditoría, y si hubiera más de una tarea, podrían escribir el mismo registro al mismo tiempo, arriesgando al paciente.

- Determine qué componentes deben ser de visibilidad estrictamente interna dentro del archivo de implementación del kernel (syscalls.cpp) y cuáles deben exponerse a través de la interfaz de cabecera pública (syscalls.h).

Interno (syscalls.cpp): pines GPIO, la ISR de sensado, la cola de solicitudes, los semáforos, la estructura syscall_req_t y KernelServiceTask todo lo que toca hardware o el mecanismo de despacho, porque nada fuera del Kernel debe manipularlo directo.

Público (syscalls.h): únicamente las funciones que la app invoca como servicio:

vvi_sys_init, sys-get-time-us, sys-wait-sensing, sys-pac-pulse, sys-log-event, sys-kick-watchdog, sys-sleep-ms.

3. Mecanismo de despacho y comunicación inter-contextos:

- Dado que el ESP32 ejecuta FreeRTOS sobre una arquitectura donde el cambio de modo de hardware puede ser emulado mediante primitivas de sincronización, justifique el uso de una **cola de solicitudes de syscalls** (Syscall Request Queue) frente a llamadas a funciones compartidas directas.

Porque una función compartida no serializa el acceso, no valida en un solo punto y no impone prioridad. La cola convierte cada solicitud en un mensaje atómico, atendido por un único hilo (KernelService) a prioridad máxima.

- ¿Qué estructura de datos (struct) permite empaquetar de forma genérica el identificador de la llamada, los parámetros variables y el canal de retorno del resultado?

`syscall-req-t` contiene id del servicio, una unión args (parámetros), una unión result (retorno) y un semaphore handle-t done-sem como canal de respuesta

- ¿Qué primitiva de sincronización (semáforo binario, cola o notificación) es la más adecuada para bloquear a la tarea solicitante hasta que el supervisor del kernel complete la operación y retorne un resultado?

Un semáforo binario (done-sem): la función publica en cola su solicitud y espera en `xSemaphoreTake`; Kernel Service lo libera con `xSemaphoreGive` al terminar.

4. Sincronización de eventos asíncronos (sensado ventricular):

- El evento de sensado de la onda R ventricular ocurre de manera asíncrona mediante una interrupción externa (ISR). ¿Por qué no es seguro realizar operaciones bloqueantes o procesamiento pesado dentro de la rutina de interrupción?

Porque ISR corre fuera del planificador con otras interrupciones a menudo enmascaradas; bloquearla (con delay o serial printf) congelaría interrupciones de igual o menor prioridad y rompera el determinismo del sistema

- Analice el flujo de sincronización: ¿Cómo debe interactuar la ISR con la tarea médica para que la llamada `sys_wait_sensing()` se desbloquee inmediatamente al detectar la despolarización intrínseca sin consumir ciclos innecesarios de CPU?

La ISR solo ejecuta `xSemaphoreGiveFromISR(sense-sem)`. Kernel Service está bloqueado en `xSemaphoreTake(sense-sem, tmrout)`, así que despierta de inmediato al llegar al flanco, sin polling ni gastos de ciclos

5. Validación defensiva y temporización en kernel:

- Establezca los criterios de validación que debe aplicar el despachador de kernel al recibir una solicitud de estimulación (`sys_pace_pulse`). Si la aplicación solicita un ancho de pulso fuera de la norma clínica ($< 100 \mu s$ o $> 2000 \mu s$), ¿cuál debe ser la respuesta del kernel para proteger al paciente sin colapsar el sistema operativo?

El despachador valida el ancho antes de leer el pin. Si está fuera de rango, registra la anomalía en el log y devuelve `success = false`, sin ejecutar el pulso físico - nunca `abort()` `esp_restart()` ni `assert()`, para no colapsar el sistema mientras protege al paciente

- Analice la relación entre el hilo supervisor y el Watchdog de hardware configurado en el arranque: ¿quién debe ser el responsable formal de alimentar el temporizador de vigilancia y bajo qué condiciones?

Unicamente el Kernel Service, mediante `esp-task-wdt-reset` al atender `SYS-KICK-WATCHDOG`. Se alimenta solo cuando la `opp` médica sigue viva y lo solicita explícitamente; si `uv-controller-task` se cuelga, deja de pedirlo y el WDT reinicia el sistema por su cuenta

4.2 Implementación en plataforma y simulación

Para la simulación del sistema, cargue en el entorno Wokwi la topología de hardware mediante el archivo `diagram.json` provisto a continuación. El pin GPIO 4 operará como entrada de sensado simulando la despolarización intrínseca (onda R), mientras que el GPIO 5 funcionará como salida de estimulación conectado al LED rojo.

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 73.4,
      "left": -96,
      "attrs": { "color": "yellow", "bounce": "0" }
    },
    { "type": "wokwi-led", "id": "led1", "top": -90, "left": 99.8, "attrs": { "color": "red" } },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 3.4,
      "left": 95.15,
      "rotate": 270,
      "attrs": { "value": "220" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "btn1:1.1", "esp:4", "yellow", [ "h-28.8", "v144", "h249.6", "v-86.4" ] ],
    [ "esp:5", "r1:1", "cyan", [ "h24.04", "v-129.55" ] ],
  ]
}
```

Syscalls (marcapasos VVI)

Capa de llamadas al sistema sobre FreeRTOS/ESP32. La lógica médica no toca GPIO, ISR ni el Task WDT: todo pasa por un despachador `KernelService` de prioridad máxima.

Cómo simular en Wokwi

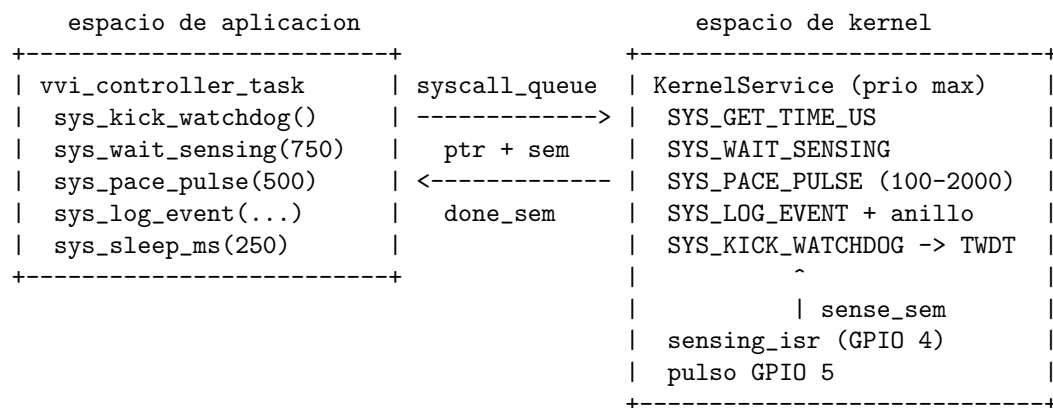
1. Abrir [un proyecto ESP32 nuevo](#).
2. Pegar `diagram.json` y `sketch.ino`.
3. Añadir los demás archivos al proyecto (`bootstrap.h`, `bootstrap.cpp`, `syscalls.h`, `syscalls.cpp`).
4. Iniciar la simulación (monitor serie a 115200).

Pin	Rol	Hardware Wokwi
GPIO 4	Sensado ventricular (onda R)	Botón amarillo a GND
GPIO 5	Pulso de estimulación	LED rojo + 220 Ω

Archivos

Archivo	Rol
<code>diagram.json</code>	Topología Wokwi (ESP32 + botón + LED)
<code>bootstrap.h/.cpp</code>	POST + WDT de la Práctica 1
<code>syscalls.h</code>	API pública
<code>syscalls.cpp</code>	Despachador, ISR, pulso, anillo de log, WDT
<code>sketch.ino</code>	Tarea médica VVI

Arquitectura



Ciclo VVI: ≈ 750 ms escuchando + (pulso 500 μ s o inhibición) + 250 ms VRP = 1000 ms entre escapes.

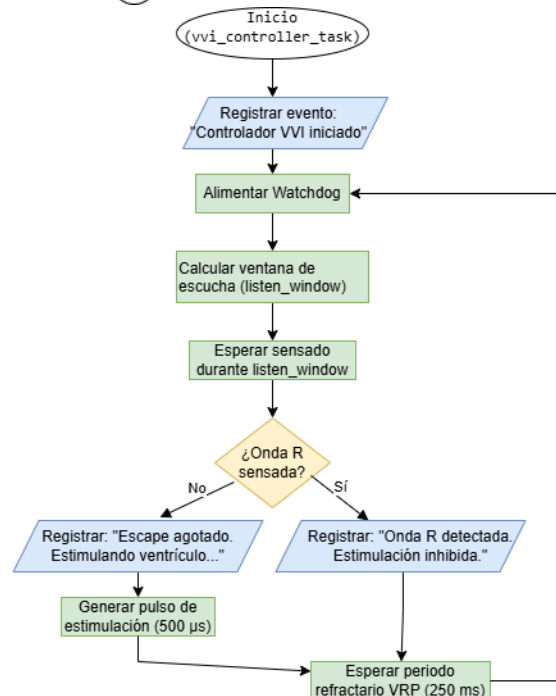
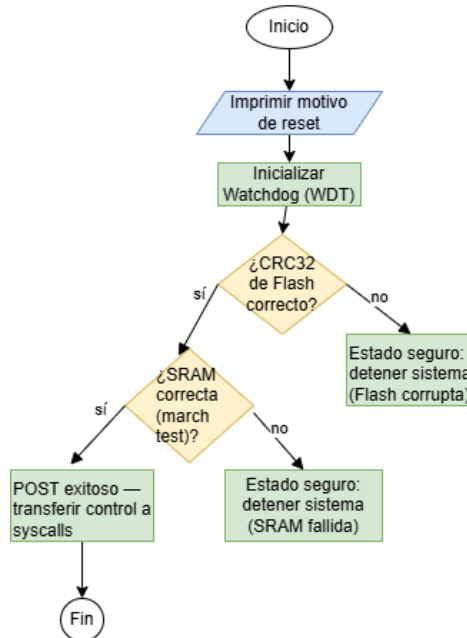
La ventana de escucha no es una constante: la tarea toma `sys_get_time_us()` al inicio de cada ciclo y resta lo que ya se consumió desde el último evento (VRP, log, pulso). Así el overhead del `printf` o del pulso no se acumula y los escapes quedan a 1 000 000 μ s exactos en las marcas `[KERNEL LOG @ ...us]`.

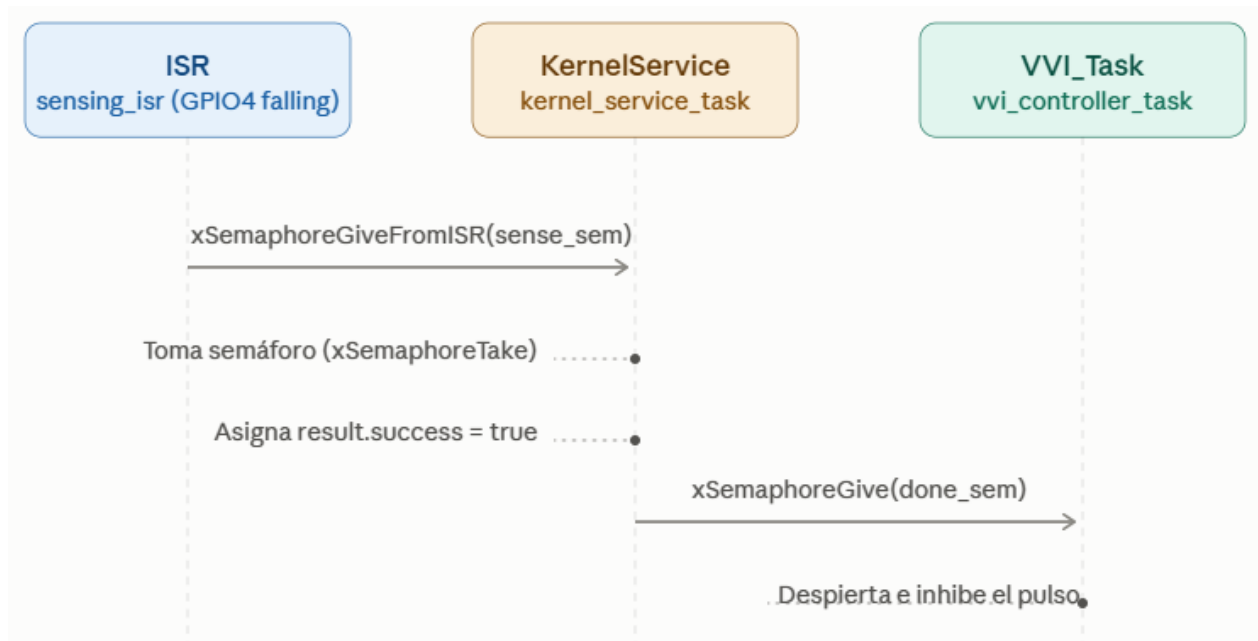
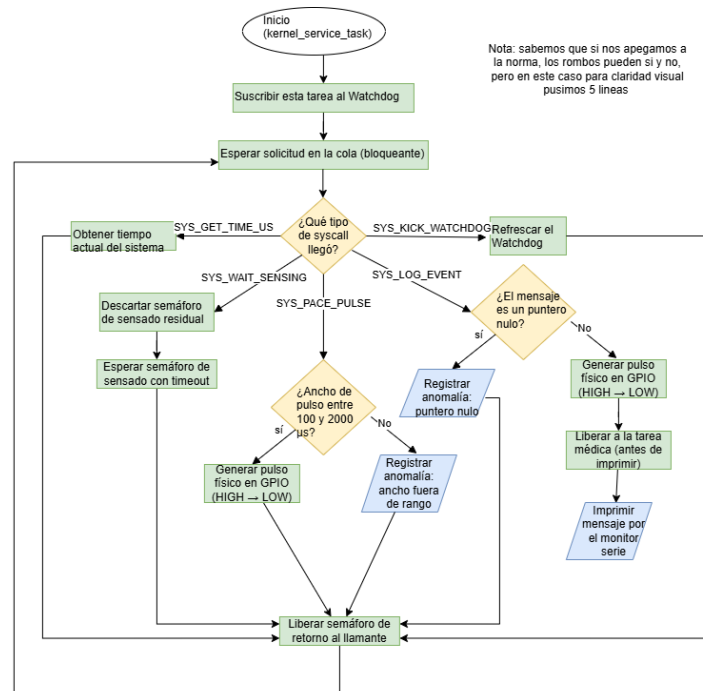
Limitación conocida

`SYS_WAIT_SENSING` bloquea al despachador hasta la onda R o el timeout (así lo pide la plantilla). Con una sola tarea médica es correcto; si hubiera más tareas de aplicación, ese servicio debería atenderse fuera del hilo `KernelService` (p. ej. entregando el semáforo directamente a la tarea solicitante) para no congelar el resto de syscalls.

Diagramas

`bootstrap_init()` (POST, CRC32, SRAM, WDT)





4.3 Pruebas (escenarios clínicos)

Repositorio: <https://github.com/MxDui/socompartido/tree/master/practica-2>

Dejar PACE_PULSE_US en 500 salvo el escenario 4.

Escenario 1 — Estimulación por escape

Acción: arrancar y no tocar el botón.

Esperado: cada ~ 1000 ms aparece `Escape agotado. Estimulando ventriculo....` El LED rojo destella $500\mu s$. El WDT no reinicia. Restar marcas `[KERNEL LOG @ ...us]` de dos escapes consecutivos: la diferencia debe acercarse a $1\,000\,000\mu s$.

Escenario 2 — Inhibición por onda R

Acción: pulsar el botón ~ 500 ms después del último evento.

Esperado: `Onda R detectada. Estimulacion inhibida.` El LED **no** enciende. El intervalo de escape se reinicia en ese instante.

Escenario 3 — Inmunidad en VRP

Acción: justo al ver un log de escape o de onda R, pulsar el botón en ráfaga.

Esperado: esos clics no generan `Onda R detectada` ni adelantan el ritmo. El kernel descarta el semáforo al abrir la siguiente ventana.

Escenario 4 — Validación defensiva

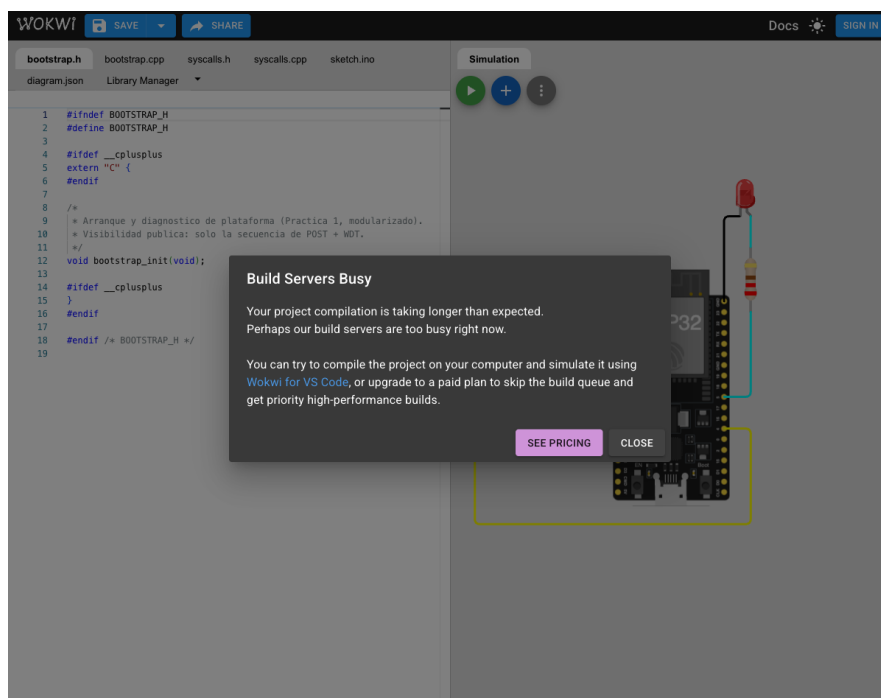
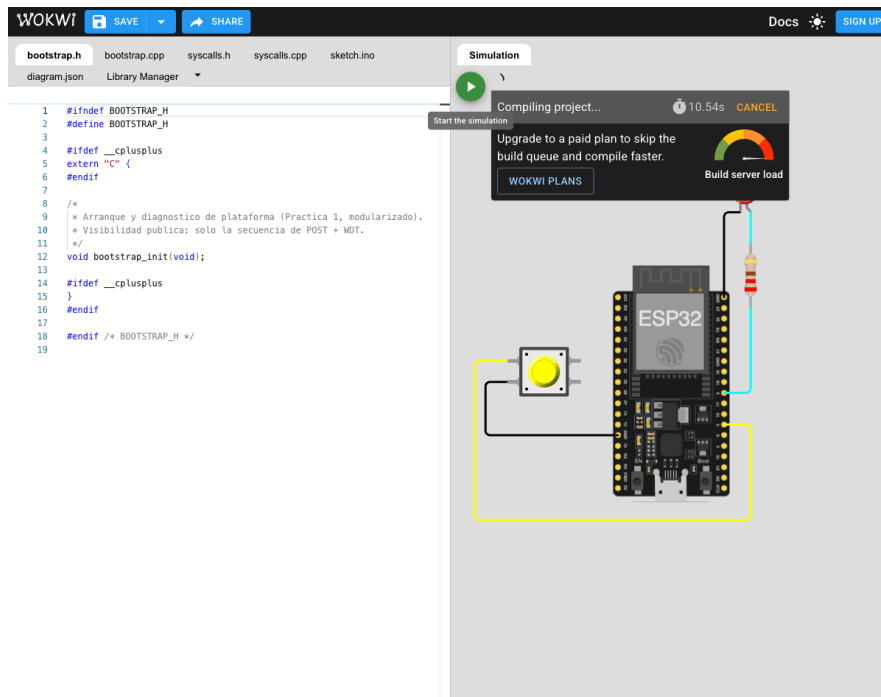
Acción: en `sketch.ino` poner `#define PACE_PULSE_US 3000` (o 50) y reiniciar.

Esperado: `[KERNEL ANOMALY] SYS_PACE_PULSE rechazado`, retorno falso, LED apagado. Restaurar 500 al terminar.

Evidencias

Las capturas de la carga del circuito y del servidor de compilación se incluyen en la sección *Evidencia*.

Evidencia



Criterio / Afirmación	1	2	3	4	5
Los conceptos y habilidades desarrollados en esta práctica son relevantes para mi formación profesional.					✓
El trabajo realizado me permitió comprender de forma práctica la teoría revisada en clase.					✓
Las actividades y problemáticas planteadas durante la sesión resultaron atractivas y motivadoras.					✓
El enfoque práctico y los retos propuestos captaron mi interés a lo largo del desarrollo.					✓
El nivel de complejidad técnica fue adecuado respecto a los conocimientos y recursos previos disponibles.					✓
La claridad de la guía permitió realizar la práctica de manera fluida y sin bloqueos innecesarios.					✓

Anexo: investigación y decisiones de diseño

Este anexo incorpora las respuestas y decisiones técnicas presentes en el reporte anterior que no estaban desarrolladas en el documento principal. Las pruebas descritas allí siguen siendo escenarios esperados; las capturas disponibles no demuestran todavía su ejecución completa.

1. Separación de privilegios

En un sistema operativo clásico, el modo usuario restringe el acceso a instrucciones privilegiadas y registros de E/S. El modo supervisor controla periféricos, interrupciones y temporizadores. En este ESP32 no hay un cambio de anillo equivalente al de una MMU de escritorio, por lo que la frontera se implementa mediante una cola de solicitudes y una tarea supervisora de prioridad máxima. La tarea médica usa solamente la interfaz de `syscalls.h`.

Si la lógica médica escribiera GPIO o registros directamente, un error en un puntero o en el ancho de pulso afectaría el hardware sin validación central. También podrían intercalarse escrituras de distintas tareas y perderse un punto único de auditoría. La capa de `syscalls` concentra la validación, serializa el acceso y devuelve un estado explícito en caso de error.

2. Llamada de función y syscall

Aspecto	Función C ordinaria	Syscall implementada
Destino	Función del mismo espacio	Despachador <code>KernelService</code>
Ejecución	<code>call/ret</code> en la misma pila	Encolado, bloqueo del llamante y cambio de tarea
Validación	Depende del programador	Obligatoria en el supervisor
Retorno	Registro o pila	Campo <code>result</code> y semáforo de finalización

La API pública llena un `syscall_req_t`, encola un **puntero** a esa solicitud y espera en `done_sem`. El kernel escribe el resultado en la estructura original y libera el semáforo. Si se encolara la estructura por valor, el kernel modificaría una copia y la tarea solicitante no recibiría el valor de retorno.

3. ISR y sincronización

La rutina de sensado no debe ejecutar `delay()`, `vTaskDelay()`, esperas largas, `Serial.printf`, asignación dinámica ni mutexes ordinarios. Una ISR no puede bloquearse como una tarea; hacerlo aumentaría la latencia de otras interrupciones. El puerto serie y el heap tampoco son operaciones apropiadas dentro de ese contexto.

La ISR solo ejecuta `xSemaphoreGiveFromISR(sense_sem)` y solicita un cambio de contexto con `portYIELD_FROM_ISR` cuando corresponde. La tarea supervisora espera con timeout y comunica el resultado a la tarea médica mediante `done_sem`. La cola lleva solicitudes desde la aplicación; los semáforos se usan para el evento asíncrono y para finalizar cada llamada.

4. Interfaz pública y componentes internos

Archivo	Responsabilidad
syscalls.h	vvi_sys_init, sys_get_time_us, sys_wait_sensing, sys_pace_pulse, sys_log_event, sys_kick_watchdog y sys_sleep_ms.
syscalls.cpp	Pines, ISR, cola, semáforos, syscall_req_t, anillo de log, kernel_service_task y generación física del pulso.
bootstrap.h/.cpp	POST de Flash/SRAM y configuración del WDT heredados de la Práctica 1.

sys_sleep_ms usa vTaskDelay en espacio de aplicación. Si el despachador durmiera durante el periodo refractario, impediría que se atendieran otras solicitudes y el watchdog.

5. Solicitud genérica y retorno

```
typedef struct {  
    syscall_id_t id;           /* servicio solicitado */  
    union { ... } args;        /* timeout / ancho / mensaje */  
    union { ... } result;      /* time_us / success */  
    SemaphoreHandle_t done_sem; /* despierta al llamante */  
} syscall_req_t;
```

La cola guarda punteros a las solicitudes, no copias. Un semáforo binario basta para indicar que terminó la operación porque el resultado ya viaja en la misma estructura. Una cola de respuesta también funcionaría, pero añadiría una segunda transferencia de datos.

6. Clasificación de requerimientos

Servicio	Entrada	Salida	Comportamiento
SYS_GET_TIME	Ninguna	Tiempo monotónico en μs	Consulta de <code>esp_timer_get_time()</code>
SYS_WAIT_SENSE	Timeout en ms	bool	Espera onda R o vence el plazo
SYS_PACE_PULSE	Ancho en μs	bool	Genera pulso solo entre 100 y 2000 μs
SYS_LOG_EVENT	Mensaje	Estado y telemetría	Copia al anillo; rechaza puntero nulo
SYS_KICK_WDT	Ninguna	Refresco del TWDT	Ejecución exclusiva del supervisor

Restricción no funcional	Criterio
Latencia de pulso	Hasta 100 μs entre solicitud y flanco de subida
Latencia de sentido	Notificación ISR a semáforo con objetivo de hasta 50 μs
Despacho	Cola y supervisor a <code>configMAX_PRIORITIES - 1</code>
Tolerancia a fallos	Parámetro inválido: anomalía en log, retorno falso, sin panic ni reinicio
Watchdog	Timeout de 3 s; solo el supervisor lo alimenta

7. Flujo de sensado y validación defensiva

```
onda R -> GPIO 4 falling -> sensing_isr
      -> xSemaphoreGiveFromISR(sense_sem)
      -> KernelService (xSemaphoreTake con timeout)
      -> req.result.success = true
      -> xSemaphoreGive(done_sem)
      -> tarea VVI despierta e inhibe el pulso
```

Antes de cada espera, el kernel drena un token residual mediante `xSemaphoreTake(sense_sem, 0)`. Así, un clic durante el periodo refractario no se conserva como una falsa onda R para la siguiente ventana.

Si `sys_pace_pulse` recibe un ancho menor que $100\ \mu\text{s}$ o mayor que $2000\ \mu\text{s}$, el kernel registra [KERNEL ANOMALY], retorna false y deja GPIO 5 intacto. Para un ancho válido, genera el pulso HIGH, espera con `delayMicroseconds` y vuelve a LOW. No ejecuta abort, `esp_restart` ni `assert` por un parámetro inválido.

8. Watchdog y alcance de la evidencia

`bootstrap_init` configura el Task WDT con timeout de 3 s. `kernel_service_task` es el único hilo suscrita y solo lo refresca al procesar `SYS_KICK_WATCHDOG`. Si la tarea médica deja de solicitar este servicio, el watchdog termina por detectar la falta de progreso.

Las capturas del documento principal muestran la carga del circuito y el estado del servidor de compilación de Wokwi. La respuesta del monitor serie, la reacción al botón, la duración efectiva del pulso y los cuatro escenarios clínicos permanecen pendientes de validación en ejecución.